

Teoria da Complexidade e Análise de Tempo de Algoritmos

Disciplina: Teoria da Computação
Professor: Daniel Bezerra

1. Descrição do Projeto

Cada equipe, composta por 3 ou 4 elementos (de preferência 4), receberá um algoritmo sorteado para implementação e análise. O objetivo é escolher duas linguagens de programação distintas (evitar linguagens muito similares, como C e C++) para implementar o algoritmo e realizar uma análise detalhada de sua complexidade de tempo.

A análise deve considerar os seguintes aspectos:

- **Descrição do algoritmo:** Problema resolvido, lógica geral e pseudocódigo.
- **Classificação assintótica:** Notação Big-O (O), Big- Ω (Omega) e Big- Θ (Theta).
- **Discussão sobre aplicabilidade:** Contextos de eficiência e limitações.
- **Simulação com dados:** Execução com diferentes tamanhos de entrada (pequena, média, grande).
- **Gráficos e Tabelas:** Comparação entre tempos medidos, complexidade teórica e desempenho entre as linguagens.
- **Análise de casos:** Melhor caso, pior caso e caso médio.
- **Reflexão final:** O algoritmo pertence à classe P? Existe uma versão NP? Há problemas semelhantes NP-completos?

2. Metodologia Experimental e Coleta de Dados

Para garantir a validade científica dos resultados, as equipes devem seguir as diretrizes abaixo:

2.1. Ambiente de Execução e Precisão

- **Hardware:** Descrever as especificações do computador utilizado (Processador, RAM, SO).
- **Isolamento:** Fechar programas em segundo plano durante os testes para evitar ruídos nas medições.
- **Precisão de Tempo:** Utilizar funções de alta precisão (ex: `System.nanoTime()` em Java, `chrono` em C++, `time.perf_counter()` em Python) em vez de tempo de relógio comum.

2.2. Protocolo de Testes (Rodadas e Casos)

A coleta de dados deve ser rigorosa para permitir análise estatística (média e desvio-padrão):

Cenário de Teste	Descrição	Nº de Execuções (por tamanho de entrada)
Melhor Caso	Entrada que favorece a lógica do algoritmo (ex: vetor já ordenado).	30 rodadas
Caso Médio	Entrada com distribuição aleatória ou típica.	30 rodadas
Pior Caso	Entrada que força o número máximo de operações (ex: vetor inversamente ordenado).	30 rodadas

Nota: As 30 rodadas devem ser repetidas para cada tamanho de entrada (pequena, média e grande) em ambos os idiomas escolhidos.

2.3. Padronização de Gráficos

- O eixo **X** deve representar o tamanho da entrada (n) e o eixo **Y** o tempo de execução.
- **Sobreposição Teórica:** É obrigatório plotar a curva da complexidade teórica esperada sobre os dados reais para demonstrar a aderência dos experimentos à teoria.
- Utilizar escalas logarítmicas quando necessário para melhor visualização de crescimentos distintos.

3. Algoritmos Disponíveis para Sorteio

- Timsort, Shell Sort.
- Algoritmo de Ukkonen, Dijkstra, A*.
- Backtracking (ex.: 8 rainhas, sudoku, etc), Subconjunto Soma (Subset Sum).
- KMP, Rabin–Karp.

4. Datas e Entregas

- A **Entrega 1** é composta pela definição das equipes através do link a seguir. Todas as equipes devem ser definidas até **08/05/2026**. Quem não estiver na planilha até essa data, terá a nota máxima **reduzida para 8**. Link da planilha de equipes:
<https://docs.google.com/spreadsheets/d/1hdlOdZupPg7BrIH4ucpENdsYB0hVVQfi/edit?usp=sharing&oid=116360864314024472071&rtpof=true&sd=true>
- A **Entrega 2** é a entrega do projeto listada a seguir, com o limite máximo até dia **16/06, às 23:59h**. Nesta entrega, deve-se enviar:
 - Slides da apresentação
 - PDF do relatório
 - Link do github na entrega no classroom ou em uma seção do relatório
- **Apresentações nos dias 17 e 22 de Junho.** A ordem será definida por sorteio e avisada com antecedência.

5. Critérios de Avaliação

Critério	Peso
Clareza e correção teórica	2.0
Análise de complexidade (Melhor, Pior, Média)	2.0
Experimentos práticos e gráficos (Aderência e Metodologia)	2.0
Código funcional e bem estruturado	2.0
Apresentação oral	2.0